Tutorial 02: Custom Bind Groups for Per-Object Data

> 💡 **Tip:** It's recommended using the 02_custom_bindgroup.html version of this tutorial as copying code works best there regarding padding and formatting. Note that the generated .html/.pdf versions can lag behind this .md file - the .md is the source of truth.
>
> ⚠️ **Build issues?** See Troubleshooting at the end of this tutorial for help reading build errors from the terminal.

In Tutorial 01, you learned the engine bind groups (Frame, Scene, Material, Object) that occupy `@group(0..3)`. Now you'll learn how to add your own custom bind group - at `@group(1)`, a slot this shader's engine roles leave free - to pass additional data to a shader, and that the engine discovers it for you by **reflecting the WGSL**, so you only name it on the C++ side.

What you'll learn: - Adding a custom bind group in a free slot (WebGPU allows only 4 bind groups per pipeline; a custom group takes an index whose engine role your shader does not use) - How the engine reflects a custom group out of your WGSL - no hand-written binding layout - Naming the group on the `ShaderDescriptor` so per-object data can target it - Implementing `preRender()` to provide per-object data via `BindGroupDataProvider` - Texture tiling and offset manipulation in shaders

What you'll build: A tiled floor with controllable tiling and offset, perfect for scrolling textures or adjusting texture scale per object.

What's provided: - Working unlit shader from Tutorial 01 - Tutorial project with scene setup - Empty `CustomRenderNode.h` class to implement

---

Understanding Custom Bind Groups

Why add custom bind groups?

The engine reserves `@group(0..3)` for its own roles: - Group 0: **Frame** - camera matrices, time (once per frame) - Group 1: **Scene** - lights, shadows, environment, clusters (once per frame) - Group 2: **Material** - textures, colors, properties (per material) - Group 3: **Object** - transform (per object)

But what if you want per-object data that's NOT a transform or material? Examples: - Scrolling water at different speeds - Individual object animations - Per-object effects or parameters

Custom bind groups solve this: WebGPU limits a pipeline to **4 bind groups** (indices 0..3 - the spec default, and Dawn's hard maximum), so there is no fifth slot for custom data. Instead, your custom group takes over an index whose engine role your shader does not use. This tutorial's unlit shader pulls in Frame (0), Material (2) and Object (3) but no Scene group, so **`@group(1)`** is free. You declare the group in WGSL and the engine reflects it - it reads the bindings, types and sizes straight out of your shader. On the C++ side you only give the group a name and a reuse policy; naming the slot in the descriptor is what tells the engine it is a custom group rather than the default engine role.

Step 1: Understanding the Project Structure

This tutorial builds directly on Tutorial 01. You should have completed the unlit shader first.

Files You'll Modify:

1. **`examples/tutorial/assets/shaders/unlit_custom.wgsl`** - Copy unlit.wgsl and extend with TileUniforms
2. **`examples/tutorial/main.cpp`** - Register new shader with custom bind group
3. **`examples/tutorial/CustomRenderNode.h`** - Implement preRender() to provide data

Starting Point:

The `main.cpp` has one commented line (similar to Tutorial 01) if it was not modified previously:

```
// floorModel->getSubmeshes()[0].material = floorMaterial->getHandle();
```

You'll uncomment this at the end once everything is ready.

Step 2: Open the Shader File

Open `examples/tutorial/assets/shaders/unlit_custom.wgsl`.

This file already contains the complete unlit shader from Tutorial 01: - VertexInput and VertexOutput structs - Frame (`@group(0)`) and Object (`@group(3)`) uniforms, pulled in via `#include "engine://core/frame_uniforms.wgsl"` and `#include "engine://core/object_uniforms.wgsl"` - Material bind group (`@group(2)`) declared inline - Vertex shader (vs_main) and fragment shader (fs_main)

You'll extend this by adding a custom bind group at `@group(1)` - the slot this shader's unused Scene role leaves free - for tiling parameters.

Step 3: Add TileUniforms Struct to Shader

Your Task:

Open `examples/tutorial/assets/shaders/unlit_custom.wgsl` and find the comment: `// Tutorial 02 - Step 3`

Add this code:

```
struct TileUniforms
{
    tileOffset: vec2f,
```

```
    tileSize: vec2f,
}
```

What it contains: - `tileOffset` - UV offset (for scrolling or shifting texture) - `tileSize` - UV scale (how many times to repeat texture)

WebGPU alignment: Each `vec2f` is 8 bytes, so total is 16 bytes - perfect for uniform buffer alignment (must be multiple of 16).

Step 4: Declare Custom Bind Group

Your Task:

In `unlit_custom.wgsl`, find the comment: `// Tutorial 02 - Step 4`

Add this code:

```
@group(1) @binding(0)
var<uniform> tileUniforms: TileUniforms;
```

Important: WebGPU allows at most **4 bind groups** per pipeline (indices 0..3) - the spec default limit, and the hard maximum on Dawn. The indices only *default* to the engine roles (Frame, Scene, Material, Object); a slot whose role your shader does not pull in is free for custom data. This unlit shader uses Frame (0), Material (2) and Object (3) but no Scene group, so `@group(1)` is available. A custom group at an index the shader already uses (here 0, 2 or 3) would collide with that engine group, and an index of 4 or higher fails pipeline creation (`bindGroupLayoutCount (5) is larger than the maximum allowed (4)`). If a shader needed all four engine roles, there would be no free slot - fold the extra data into the material group instead. The engine reflects whatever you declare here.

Bind group slots: - Group 0: **Frame** (engine role) - pulled in via `#include` when your shader needs camera data - Group 1: **Scene** (engine role) - *unused by this shader*, so our custom `TileUniforms` takes it - Group 2: **Material**, Group 3: **Object** (engine roles this shader uses)

Each group can have multiple bindings (0, 1, 2...) for different resources within that group.

Step 5: Modify Fragment Shader to Use TileUniforms

Your Task:

In `unlit_custom.wgsl`, find the comment: `// Tutorial 02 - Step 5`

Update the fragment shader to apply tiling before texture sampling:

```
@fragment
fn fs_main(input: VertexOutput) -> @location(0) vec4f {
    // Apply tiling and offset to UV coordinates
    let tiledUV = input.texCoord * tileUniforms.tileSize + tileUniforms.tileOffset;

    // Sample texture with modified UVs
    let textureColor = textureSample(baseColorTexture, textureSampler, tiledUV);
    let finalColor = textureColor * unlitMaterialUniforms.color;
    return finalColor;
}
```

What this does: 1. `input.texCoord` - Original UV from mesh (0-1 range) 2. `* tileUniforms.tileSize` - Scale UV (e.g., ×4 = repeat texture 4 times) 3. `+ tileUniforms.tileOffset` - Shift UV (e.g., +0.5 = scroll halfway) 4. Sample texture at modified coordinates

Example values: - `tileSize = (2, 2)` - Texture repeats 2×2 times across surface - `tileOffset = (0.5, 0)` - Texture shifted 50% to the right

---

Step 6: Register Shader with Custom Bind Group

Your Task:

Open `examples/tutorial/main.cpp` and find the comment: `// Tutorial 02 - Step 6`

The shader is registered with a `ShaderDescriptor` - a small declaration the engine fills out by reflecting the WGSL. You don't list bindings or sizes; reflection recovers them from the `@group(1)` block you wrote. You only have to do two things:

1. Point the descriptor at the new shader file
2. Name the custom group so per-object data can target it

Point the Descriptor at the New Shader:

In the descriptor, change the path line to the tiling shader:

```
unlitShader.path = PathProvider::getShaders("unlit_custom.wgsl"); // was "unlit.wgsl"
```

The Frame, Material and Object groups are still discovered automatically (Frame/Object from the `#includes`, Material from the structs the shader declares), so nothing else about those changes.

Name the Custom Group:

Right after the `material` group is assigned, add the custom group entry:

```
unlitShader.groups[1] = {
    "TileUniforms",                                 // Group name (targeted from preRe
    engine::rendering::BindGroupType::Custom,       // Mark as a custom group
    engine::rendering::BindGroupReuse::PerObject,   // Cache one per object
    {}                                              // No per-binding overrides - refl
};
```

Naming the entry is what claims the slot: an unnamed `@group(1)` would fall back to the engine's Scene role, but a named descriptor entry defines a custom group at any index.

The registration call below it is already in place and stays the same:

```
shaderRegistry.registerShader(shaderFactory.buildFromDescriptor(unlitShader));
```

What you're telling the engine: - The group at `@group(1)` is called `"TileUniforms"` (your `preRender()` will target this name) - It's a custom group, cached per object - The empty `{}` means "no overrides" - the binding, its type and its size all come from reflecting the shader

How the binding happens:

> 📝 **Note:** `buildFromDescriptor()` expands the `#includes`, reflects the resulting WGSL, and builds the bind group layouts from what it finds. For `@group(1)` it reads your `TileUniforms` declaration directly - the binding index, the fact that it's a uniform, and its 16-byte size all come from the shader. The descriptor only supplies the things WGSL can't express: the engine-facing name and the reuse policy.
>
> **Advanced Readers:** See WgslReflector.cpp for the reflection pass and WebGPUShaderFactory.cpp (`buildFromDescriptor`) for how the reflected groups become bind group layouts.

Why reflection instead of hand-listing bindings? - **The shader is the source of truth**: you declared `TileUniforms` once in WGSL; reflection reuses that rather than making you restate the size and bindings in C++ (which could drift) - **Less to get wrong**: no manual byte counts or stage flags for the custom group - **Still explicit where it matters**: you choose the group name and reuse policy, which are engine concepts the shader can't express

---

Step 7: Implement CustomRenderNode

Your Task:

Open `examples/tutorial/CustomRenderNode.h` and find the comment: `// Tutorial 02 - Step 7`

The struct is already defined:

```
struct TileUniforms
{
    glm::vec2 tileOffset;
    glm::vec2 tileSize;
};
```

Implement the `preRender()` method:

```
virtual void preRender(std::vector<engine::rendering::BindGroupDataProvider> &outProvi
{
    auto dataProvider = engine::rendering::BindGroupDataProvider::create(
        "unlit",      // Shader name (must match registration)
        "TileUniforms",      // Bind group name (must match groups[1] in registration)
        tileUniforms,        // Uniform data (struct instance)
        engine::rendering::BindGroupReuse::PerObject,  // Cache per object
        getId()              // Instance ID (node's unique ID)
    );
    outProviders.push_back(dataProvider);
}
```

Understanding preRender():

This method is called by the scene graph before rendering each frame. It's your opportunity to provide updated data to the GPU.

BindGroupDataProvider::create() parameters: 1. **Shader name** - Which shader needs this data 2. **Bind group name** - Which bind group in that shader (the name you gave `groups[1]`) 3. **Data** - Actual uniform data (will be copied to GPU) 4. **Reuse policy** - When to rebind (PerObject means cache per unique object) 5. **Instance ID** - Unique identifier for caching (use node ID for per-object)

Why PerObject reuse? If you have 10 floor tiles with different tiling, each gets its own cached bind group. The renderer only rebinds when switching between different objects.

Step 8: Create CustomRenderNode Instance

Your Task:

Open `main.cpp` and find the comment: `// Tutorial 02 - Step 8`

Update the code to use CustomRenderNode:

```
// Change from ModelRenderNode to CustomRenderNode
auto floorNode = std::make_shared<demo::CustomRenderNode>(floorModel);
floorNode->tileUniforms.tileOffset = glm::vec2(0.2f, 0.0f);
floorNode->tileUniforms.tileSize = glm::vec2(4.0f, 4.0f);
floorNode->getTransform().setLocalScale(glm::vec3(10.0f, 1.0f, 10.0f));
rootNode->addChild(floorNode);
```

What this does: - Creates custom node with floor model - Sets tileOffset to shift texture 20% to the right - Sets tileSize to 4×4 (repeat texture 16 times total) - Scales floor physically to 10×10 units

---

Step 9: Uncomment Material Assignment

Your Task:

Open `main.cpp` and find the comment: `// Tutorial 02 - Step 9`

Uncomment the material assignment line:

```
// Uncomment this line:
floorModel->getSubmeshes()[0].material = floorMaterial->getHandle();
```

Why uncomment now? Now that the shader is complete and registered, the engine can create the render pipeline successfully.

Step 10: Build and Run

```
# Windows
scripts\build-example.bat tutorial Debug WGPU

# Linux
bash scripts/build-example.sh tutorial Debug WGPU
```

VS Code: Press `F5` to build and run.

Where the executable lands: build directories are per backend. On Windows a `WGPU` build goes to `examples\build\tutorial\Windows\Debug-WGPU` (a `DAWN` build uses `Debug-DAWN`; only a Dawn-from-source build - fourth argument `SOURCE` - uses the plain `Debug` folder, and Emscripten/`EMDAWN` builds go under `Emscripten\Debug`). On Linux/Mac the build directory is `examples/build/tutorial/<Linux|Mac>/Debug`.

Expected Result

You should see: - ✅ **Floor** with tiled cobblestone (4×4 repetitions) - ✅ **Texture slightly offset** to the right (20% shift) - ✅ **Fourareen object** with PBR lighting (unchanged)

Compare to Tutorial 01: - Tutorial 01: Texture stretched across entire floor (1×1) - Tutorial 02: Texture repeated 4×4 times with smaller tiles

---

Understanding the Data Flow

Lifecycle of custom bind group data:

```
1. Scene graph traversal:
   └─ Scene calls preRender() on each node

2. CustomRenderNode::preRender():
   ├─ Reads tileUniforms member variable
   ├─ Creates BindGroupDataProvider with data
   └─ Adds provider to outProviders vector

3. Scene collects all providers:
   └─ Passes them to Renderer::renderFrame()

4. Renderer processes providers:
   ├─ FrameCache::processBindGroupProviders() creates GPU buffers
   ├─ Caches bind group by (shaderName, bindGroupName, instanceId)
   └─ Uploads data to GPU via writeBuffer()

5. During rendering:
   ├─ BindGroupBinder::bind() checks cache
   ├─ Finds cached bind group by instanceId
   └─ Binds it at the correct index (in this case @group(1)) in the shader

6. Shader execution:
   └─ Fragment shader reads tileUniforms.tileOffset and tileSize
```

Key insight: You provide data in `preRender()`, the engine handles caching and GPU upload automatically.

Why preRender() Works - The Timing:

Remember WebGPU's command recording model: 1. **Scene Traversal**: Scene calls `preRender()` on all nodes \u2192 Collects bind group providers 2. **GPU Upload**: Engine processes providers \u2192 Creates buffers \u2192 Writes data with `queue.writeBuffer()` 3. **Command Recording**: Engine records draw commands \u2192 Binds groups \u2192 Issues `draw()` 4. **Submission**: Complete command buffer sent to GPU queue 5. **GPU Execution**: Commands execute, reading the fresh data you provided

This is why updating `tileUniforms` in `preRender()` works - the data reaches the GPU before the draw command executes!

Understanding BindGroupReuse Policies

Why PerObject for TileUniforms?

The reuse policy determines when bind groups are cached and rebound:

Policy	When it Changes	Example Use Case
Global	Never (constant for whole frame)	Default textures, global settings
PerFrame	Once per frame	Camera, time, global lighting
PerObject	Per object	Object transforms, per-object parameters
PerMaterial	Per material	Material textures, material properties

For custom bind groups: - Use `PerObject` when data is unique per object instance (like our tiling) - Use `PerFrame` if data updates every frame but is shared (like a global animation time) - Use `PerMaterial` if data is shared by objects with same material

Performance impact: - `PerObject` with unique data = creates many bind groups (one per object) - `PerObject` with shared data = reuses cached bind groups automatically

The engine's `FrameCache` handles the caching and `BindGroupBinder` the binding - you just specify the policy.

Experiments to Try

1. Animate the offset - In `CustomRenderNode`, add an `update()` method:

Implement the `UpdateNode` behavior.

```
class CustomRenderNode : public engine::scene::nodes::ModelRenderNode, public engine::
```

Override the default `update(float deltaTime)` method.

```
virtual void update(float deltaTime) override {
    tileUniforms.tileOffset.x += deltaTime * 0.1f; // Scroll right
}
```

2. Different tiling per object - Create multiple floor nodes:

```
auto floor1 = std::make_shared<demo::CustomRenderNode>(floorModel);
floor1->tileUniforms.tileSize = glm::vec2(2.0f, 2.0f);
```

```
floor1->getTransform().setLocalPosition(glm::vec3(-5.0f, 0.0f, 0.0f));
floor1->getTransform().setLocalScale(glm::vec3(5.0f, 1.0f, 5.0f));
rootNode->addChild(floor1);

auto floor2 = std::make_shared<demo::CustomRenderNode>(floorModel);
floor2->tileUniforms.tileSize = glm::vec2(8.0f, 8.0f);
floor2->getTransform().setLocalPosition(glm::vec3(5.0f, 0.0f, 0.0f));
floor2->getTransform().setLocalScale(glm::vec3(5.0f, 1.0f, 5.0f));
rootNode->addChild(floor2);
```

3. Add rotation - Extend TileUniforms:

Adjust C++ Struct

```
// CustomRenderNode.h
struct TileUniforms
{
    glm::vec2 tileOffset = glm::vec2(0.0f);
    glm::vec2 tileSize = glm::vec2(1.0f);
    glm::vec4 rotation = glm::vec4(0.0f);
    // Rotation in .x, padding in .yzw for 16-byte alignment
};
```

Adjust WGSL Struct

```
struct TileUniforms {
    tileOffset: vec2f,   // Offset
    tileSize: vec2f,     // Scale
    rotation: vec4f,     // rotation.x = angle in radians, yzw = padding
}
```

Define WGSL Helper Function

```
fn rotate2D(uv: vec2f, angle: f32) -> vec2f {
    let cosA = cos(angle);
    let sinA = sin(angle);
    return vec2f(
        uv.x * cosA - uv.y * sinA,
        uv.x * sinA + uv.y * cosA
    );
}
```

Update Fragment Shader

```
let rotatedUV = rotate2D(input.texCoord - 0.5, tileUniforms.rotation.x) + 0.5;
let tiledUV = rotatedUV * tileUniforms.tileSize + tileUniforms.tileOffset;
```

Update cpp code to set a rotation of your choice

```
floorNode->tileUniforms.rotation = glm::vec4(0.5f, 0.0f, 0.0f, 0.0f);
```

Understanding WebGPU Custom Resources

WebGPU bind group creation:

When you call `BindGroupDataProvider::create()`, the engine internally:

1. **Creates GPU buffer** - Allocates uniform buffer with `wgpu::BufferUsage::Uniform | CopyDst`
2. **Writes data** - Copies your struct to GPU with `queue.writeBuffer()`
3. **Creates bind group** - Links buffer to bind group layout via `device.createBindGroup()`
4. **Caches by key** - Stores in `FrameCache::customBindGroupCache[key]` where key = (shader, bindGroup, instanceId)

Next frame: - Same object? Reuse cached bind group, just update buffer if data changed - Different object? Create new bind group or fetch from cache by different instanceId

Memory management: - Bind groups live in `FrameCache::customBindGroupCache` and **persist across frames** - each one is created exactly once, then only its buffer contents are refreshed via `updateBuffer()` - The per-frame cleanup (`FrameCache::endFrame()`) drops only truly per-frame data (lights, shadow requests, render targets); the custom bind group cache survives it - The cache is only emptied by the full `FrameCache::clear()`, which runs on scene changes

Buffer sizing - the capacity is fixed at creation: - The auto-created buffer behind your custom group is sized at the layout's `minBindingSize` - exactly the size of the struct you declared in WGSL (for a runtime-sized array binding, that is ONE array element) - `WebGPUBindGroup::updateBuffer()` **clamps** any write that would exceed the buffer size and logs a warning (`updateBuffer: write ... exceeds buffer ... - clamping`) - the excess bytes are dropped, not uploaded - A fixed-size struct like our 16-byte `TileUniforms` never hits this. If you ever provide **growing** data (e.g. a runtime-sized array), preallocate a buffer at full capacity and hand it to the factory as an override - see how `DebugPass` does it in DebugPass.cpp (`bufferFactory().createStorageBuffer(...)` at max capacity, then a `BindGroupResource` override passed into `createBindGroup()`) - or keep the data fixed-size

---

Key Takeaways

✅ **Custom Bind Groups** - Extend shader capabilities with custom data in a free bind group slot, discovered by reflection
✅ **BindGroupDataProvider** - Simple API to pass CPU data to GPU shaders
✅ **preRender() Lifecycle** - Called before rendering, perfect for per-frame updates
✅ **Reuse Policies** - Control caching behavior (PerFrame, PerObject, PerMaterial)
✅ **Node Customization** - Extend ModelRenderNode to add custom shader data

WebGPU Concepts Learned: - WebGPU's 4-bind-group limit, and reusing a free index for custom data - Custom uniform buffer creation and caching - Alignment requirements for uniform structs (16-byte boundaries) - Efficient resource reuse via caching policies

---

What's Next?

In **Tutorial 03**, you'll write a glass shader and learn: - The Scene bind group (`@group(1)`): lights and shadow maps via `#include` - Fresnel-style rim lighting and a specular highlight (view-vector math) - Receiving shadows with the engine's shared `calculate_shadow()` (cascades, bias, PCF) - Transparency: the `Transparent` feature flag, alpha blending, and why transparent and custom-shader materials render back-to-front in the forward pass

Next Tutorial: 03_glass_shader.md

Further Reading

- Bind Group System Documentation
- Node Type System Guide
- WebGPU Bind Group Spec
- Tutorial 01: Unlit Shader

Troubleshooting

Build Failures - Reading Terminal Output

⚠️ **Important:** When building via the VS Code task (which runs `scripts/build-example.bat` / `build-example.sh`), the task system may report success even if the build actually failed. You **MUST check the terminal output** to see the real result.

What to look for in terminal: 1. Scroll to the **very end** of the terminal output 2. Look for `[SUCCESS] Example 'tutorial' built successfully!` - if this appears, build succeeded 3. If you see `[ERROR] Build failed.` (or `[ERROR] CMake configuration failed.`) - the build failed regardless of task status

Common build issues: - **Missing semicolons** - WGSL requires `;` at end of statements - **Struct size mismatch** - `tileUniforms` size must be 16 bytes (2 × vec2f) - **Bind group naming** - the `"TileUniforms"` name in `groups[1]` must match the group your `preRender()` targets; the `@group(1)` index in the shader must match the `groups[1]` key in the descriptor - **CMake cache** - Delete the example's build folder (`examples\build\tutorial`) then rebuild clean

Bind Group Issues

"Unable to find bind group 'TileUniforms'" - Check shader name matches in `preRender()` ("unlit") and registration as well as material assignment - Ensure bind group name matches exactly (case-sensitive) between `groups[1]` and `preRender()` - Verify the custom group's index does not collide with an engine group the shader actually uses (here 0, 2 and 3)

"bindGroupLayoutCount (5) is larger than the maximum allowed (4)" / "BindGroupBinder: group index 4 exceeds wgpu's max of 4" - The custom group is declared at `@group(4)` or higher - WebGPU pipelines are limited to 4 bind groups (indices 0..3). Move the custom group into a slot the shader's engine roles leave free (for this shader, `@group(1)`), and key the descriptor entry accordingly (`groups[1]`)

Floor appears stretched/wrong - Verify TileUniforms struct size matches shader (16 bytes = 2 × vec2f) - Check reuse policy is `PerObject` in both registration and `preRender()`

Data doesn't update - Ensure `preRender()` is called (check node is enabled and in scene graph) - Verify you're modifying the member variable, not a local copy - Check instance ID is correct in `BindGroupDataProvider::create()` - A log warning `updateBuffer: write ... exceeds buffer ... - clamping` means your C++ struct is larger than the WGSL struct - the write is clamped to the GPU buffer's size, so the tail of your data never reaches the shader

Shader compilation error - Ensure the custom group's index is not one the shader's engine roles already occupy (0, 2, 3 here) and stays below WebGPU's limit of 4 bind groups - Ensure TileUniforms struct is defined before use - In Debug builds the shader validator is fatal, so the load fails fast with the offending line - Verify there are no missing `;`

Debug Strategy

If errors are unclear: 1. Run the tutorial - shader reflection and validation happen at load time 2. Check the console / `run_out.log` - shader compile and validation errors are printed there with the line/column 3. In Debug builds a malformed custom group (wrong index, redeclared engine struct) fails the load with a structured diagnostic